

# lab3-2 基于 UDP 服务设计可靠传输协议并编程实现

---

才子桓 2212700 密码科学与技术

## 实验要求

---

**实验3-1：** 利用数据报套接字在用户空间实现面向连接的可靠数据传输，功能包括：建立连接、差错检测、确认重传等。流量控制采用停等机制，完成给定测试文件的传输。

**实验3-2：** 在实验 3-1 的基础上，将停等机制改成基于滑动窗口的流量控制机制，采用固定窗口大小，支持累积确认，完成给定测试文件的传输。

**实验3-3：** 在实验 3-2 的基础上，选择实现一种拥塞控制算法，也可以是改进的算法，完成给定测试文件的传输。

**实验3-4：** 基于给定的实验测试环境，通过改变网路的延迟时间和丢包率，完成下面3组性能对比实验：

- (1) 停等机制与滑动窗口机制性能对比；
- (2) 滑动窗口机制中不同窗口大小对性能的影响；
- (3) 有拥塞控制和无拥塞控制的性能比较。

要求如下：

1. 实现单向传输。
2. 对于每一个任务要求给出详细的协议设计。
3. 给出实现的拥塞控制算法的原理说明。
4. 完成给定测试文件的传输，显示传输时间和平均吞吐率。
5. 性能测试指标包括吞吐率、文件传输时延等，给出图形结果并进行分析。
6. 完成详细的实验报告（每个任务完成一份）。
7. 编写的程序应该结构清晰，具有较好的可读性。
8. 提交程序源码、可执行文件和实验报告。

## 实验说明

---

本次实验在实验3-1的基础上，将停等机制改成基于滑动窗口的流量控制机制，发送端 Client 为 32 大小的滑动窗口，接收端 Server 窗口大小为 1。累计确认采用 GBN 协议。连接建立与断开的方式不变。

## 实验环境

---

IDE: Visual Studio 2022

语言: c++ 14标准

lab3 测试文件

路由器 Router

本实验在 Windows 系统下进行。

## 实验分析及代码实现

---

这里仅展示对比 3-1 中改动的部分。序列号 Seq 仍采用实验 3-1 的编号方式，差错检验、连接与断连协议与 3-1 实验一致。这里便不做赘述。

# 滑动窗口和累计确认

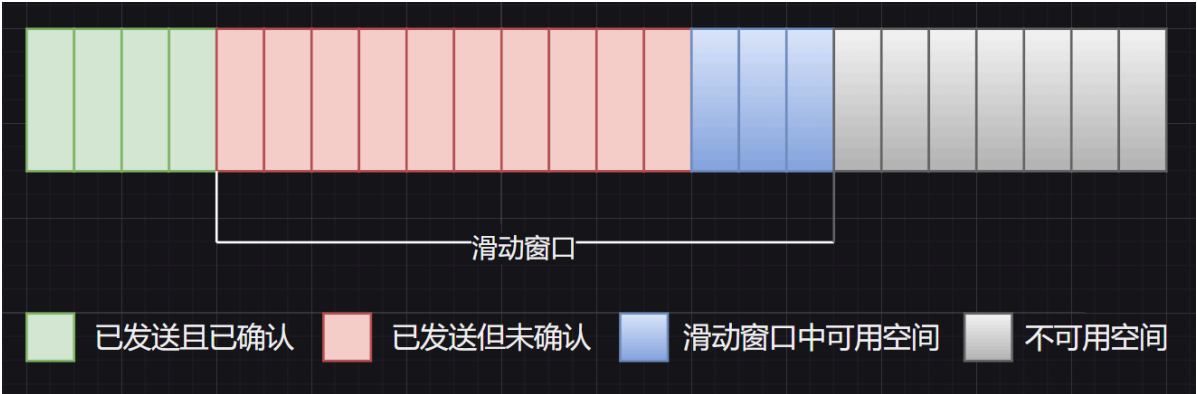
根据实验要求，这里我设置发送端 Client 为 32 大小的滑动窗口，接收端 Server 窗口大小为 1。

在 3-1 的实验中，我们采用停等协议，发送端 Client 发送数据后，等待接收端 Server 的确认。接收端 Server 收到数据后，发送确认信息。发送端 Client 收到确认信息后，发送下一条数据。这样明显传输效率过慢，传输时间过长。

所以在 3-2 实验改进时，我们采用滑动窗口协议。

发送端 Client 发送数据时，首先将数据放入发送窗口，然后发送窗口右移一位。发送窗口满时，发送端 Client 等待接收端 Server 的确认。接收端 Server 收到数据后，将数据放入接收窗口，然后发送确认信息。发送端 Client 收到确认信息后，将发送窗口左移一位。发送窗口有空时，发送端 Client 可以继续发送数据。

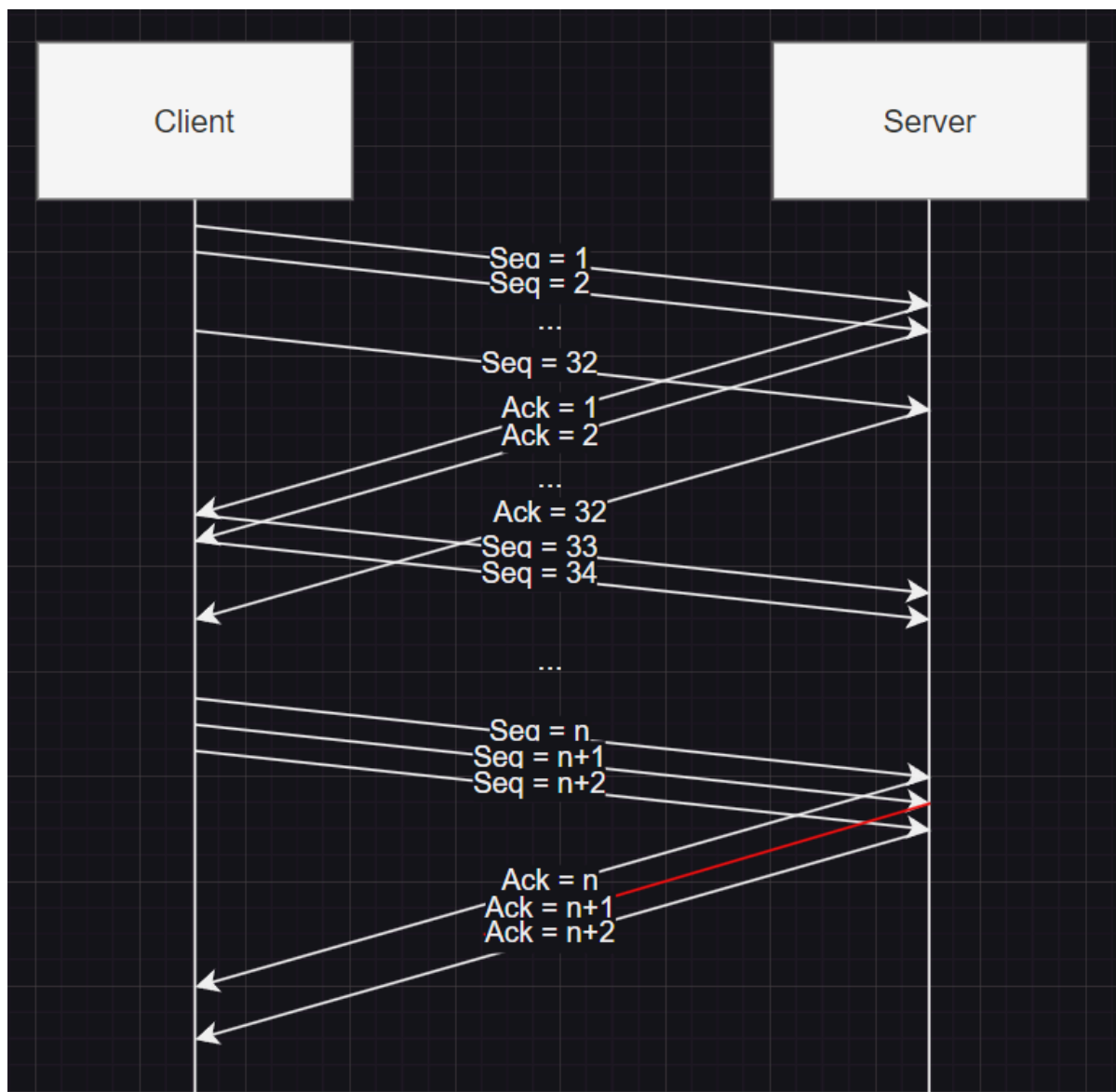
滑动窗口示意图：



这里我们采用 GBN 的累计确认协议。

累计确认机制主要是应对 ACK 报文丢失的问题，根据实验要求 Server 窗口大小为 1，确保接收数据报文是按序接收。累计确认保证只要 Client 收到了一个 ACK 报文，就说明前面的数据报文都被正确接收到了。如果发生了 ACK 报文丢失，Client 仍可以通过后面发送的 ACK 报文去确认之前所有的报文已成功被 Server 接收。

累计确认示意图：



GBN 协调称回退 N 帧协议，在连续发送数据帧时，当检测到失序或错误时，要求重传最后正确接收帧后的所有未确认帧。如果超时未收到确认，将重传最近 N 个帧。接收端只按序接受，即使接收到正确但不按序的帧也会丢弃，并重复发送确认帧以防确认丢失。

## 超时重传，快速重传

这里相对于实验 3-1，取消了最大传输次数。

**快速重传：**当 Client 丢失数据包 n 时，当后续数据包 n+1，n+2 和 n+3 到达时，由于累计确认机制，Server 会都返回 ACK 为 n 的报文。当 Client 接收到三次一样的 ACK 报文，会触发快速重传机制，将丢失的数据包 n 以及后面已发送的数据包重传。

**超时重传：**Client 每发送一个数据包都会为这个数据包设置一个定时器，当定时器超时后，Client 会重传这个数据包以及后面已发送的数据包。

## 相关代码修改

本次实验修改主要 Client 中。

首先添加一些在滑动窗口中需要的常量和变量。

```
// 滑动窗口大小
#define window_Size 32

// 滑动窗口的开始和到达
int start = 0;
```

```

int arrive = 0;
// 计时
int messagestart;
// 标志位，标记是否需要重传和是否完成
bool finish = false;
bool resend = false;

// 定义缓冲区存已发送未确认的 message
queue<Message> messageBuffer;

```

本次实验相对于实验 3-1，本次实验采用滑动窗口机制，Client 在发送的同时需要接收来自 Server 的 ACK 报文。所以为了防止 Client 出现程序冲突，采用了多线程机制。这里我们需要创建两个进程，一个进程用来发送数据，一个进程用来接收 ACK 报文。通过多线程，Client 能够并行地处理数据的发送、接收，能够在处理网络通信时提高效率，减少等待时间，避免阻塞，更加灵活和响应迅速。

所以需要添加一个新的结构体 `parameters`，定义创建 ACK 接收线程时候传过去的参数，由于创建线程只能传一个参数，所以这里将需要传过去的信息绑定成结构体。

同时定义一个辅助锁，确保多线程环境下的线程安全，防止多个线程同时访问共享资源时出现数据竞争或不一致的情况。

```

struct parameters {
    SOCKET clientSocket;
    SOCKADDR_IN serverAddr;
    int nummessage;
};

// 辅助锁
HANDLE mutex;

```

缓冲区更新函数 `updateBuffer`，收到了 ACK 报文后，则需要把缓冲区那些 Seq 值小于等于 ACK 报文的 ack 值的那些 message 弹出队列。即 Server 已确认接收，删掉 Client 的缓存。

```

void updateBuffer(int ackNum) {
    while (!messageBuffer.empty()) {
        const Message& frontMsg = messageBuffer.front();
        if (frontMsg.SeqNum <= ackNum) {
            messageBuffer.pop(); // 删除已确认的消息
        }
        else
            break; // 一旦遇到一个未确认的消息，停止循环
    }
}

```

接收 Ack 的线程函数 `recvackthread`，从 Server 接收到的 ACK 消息，并根据这些消息更新发送窗口的状态。

函数先创建一个互斥锁 `mutex`，用于保护共享资源。声明一个 `Message` 结构体变量 `recvMsg`，用于接收消息，从 Client 接收消息，并将接收到的消息存储在 `recvMsg` 结构体中。如果接收到的字节数大于 0 且有效、ACK 编号大于等于起始编号，则等待并获取互斥锁，然后更新发送窗口的缓冲区，更新起始编号，并释放互斥锁。

如果发送窗口的起始编号不等于到达编号，则更新消息开始时间。

如果接收到的 ACK 编号等于消息数量加 1，则表示所有消息都已被确认，设置完成标志 `finish` 为 `true`。

如果接收到的 ACK 编号与之前的错误 ACK 编号不同，则重置错误计数 `errorcount` 为 0，并更新错误 ACK 编号 `errorack`。

如果接收到的 ACK 编号与之前的错误 ACK 编号相同，则增加错误计数 errorcount。如果错误计数达到 3，则设置重发标志 resend 为 true。

```
DWORD WINAPI recvackthread(PVOID useparameter) {
    mutex = CreateMutex(NULL, FALSE, NULL);
    if (mutex == NULL) {
        Print("CreateMutex failed.");
        return 1;
    }

    parameters* p = (parameters*)useparameter;
    if (p == NULL) {
        Print("Invalid parameters.");
        CloseHandle(mutex);
        return 1;
    }

    SOCKADDR_IN serverAddr = p->serverAddr;
    SOCKET clientSocket = p->clientSocket;
    int nummessage = p->nummessage;
    int AddrLen = sizeof(serverAddr);

    int errorack = -1;
    int errorcount = 0;

    unsigned long mode = 1;
    if (ioctlsocket(clientSocket, FIONBIO, &mode) == SOCKET_ERROR) {
        Print("ioctlsocket failed.");
        CloseHandle(mutex);
        return 1;
    }

    while (1) {
        Message recvMsg;
        int recvByte = recvfrom(clientSocket, (char*)&recvMsg, sizeof(recvMsg),
0, (sockaddr*)&serverAddr, &AddrLen);

        if (recvByte > 0) {
            if (recvMsg.check()) {
                if (recvMsg.AckNum >= start) {
                    WaitForSingleObject(mutex, INFINITE);
                    updateBuffer(recvMsg.AckNum);
                    start = recvMsg.AckNum + 1;
                    cout << Get_Time() << " " << "[Receive] ack = " <<
recvMsg.AckNum << endl;
                    cout << Get_Time() << " " << "[window] window Residual
Size: " << (window_Size - (arrive - start)) << ", has sent but not received
confirmation: " << (arrive - start) << endl;
                    ReleaseMutex(mutex);
                }
                if (start != arrive)
                    messagestart = clock();

                if (recvMsg.AckNum == nummessage + 1) {
                    finish = true;
                }
            }
        }
    }
}
```

```

        return 0;
    }

    if (errorack != recvMsg.AckNum) {
        errorcount = 0;
        errorack = recvMsg.AckNum;
    }
    else
        errorcount++;

    if (errorcount == 3)
        resend = true;
}
}
}
CloseHandle(mutex);
return 0;
}

```

主函数的处理部分。Socket 初始化和计算大小等过程与 3-1 一样。Client 发送数据包，同时窗口开始计数，计时器开始工作。如果窗口未满，继续发送，同时会递增 arrive。当超时或者触发快速重传的时候，会利用基于队列的缓冲区进行遍历重传，同时每个数据报文重传后会重新加到队尾，这样如果重传一直不成功可以实现无限次数重传。

```

int batchNum = fileSize / Max_DATA_SIZE;
int leftNum = fileSize % Max_DATA_SIZE;
int nummessage;

if (leftNum != 0)
    nummessage = batchNum + 2;
else
    nummessage = batchNum + 1;

parameters useparameter;
useparameter.serverAddr = serverAddr;
useparameter.clientSocket = clientSocket;
useparameter.nummessage = nummessage;
HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)recvackthread,
&useparameter, 0, 0);

int count = 0;
while (1) {
    if (arrive < start + window_size && arrive < nummessage + 2) {
        Message datamessage;
        if (arrive == 2) {
            datamessage.SrcPort = ClientPORT;
            datamessage.DestPort = RouterPORT;
            datamessage.size = fileSize;
            datamessage.flag += FileName;
            datamessage.SeqNum = arrive;

            for (int i = 0; i < realname.size(); i++)
                datamessage.data[i] = realname[i];
            datamessage.data[realname.size()] = '\0';
            datamessage.setCheck();
        }
    }
}

```

```

    }
    else if (arrive == batchNum + 3 && leftNum > 0) {
        datamessage.SrcPort = ClientPORT;
        datamessage.DestPort = RouterPORT;
        datamessage.SeqNum = arrive;
        for (int j = 0; j < leftNum; j++)
            datamessage.data[j] = fileBuffer[batchNum * Max_DATA_SIZE +
j];

        datamessage.setCheck();
    }
    else {
        datamessage.SrcPort = ClientPORT;
        datamessage.DestPort = RouterPORT;
        datamessage.SeqNum = arrive;
        for (int j = 0; j < Max_DATA_SIZE; j++)
        {
            datamessage.data[j] = fileBuffer[count * Max_DATA_SIZE + j];
        }
        datamessage.setCheck();
        count++;
    }
    if (start == arrive)
        messagestart = clock();

    WaitForSingleObject(mutex, INFINITE);
    messageBuffer.push(datamessage);
    sendto(clientSocket, (char*)&datamessage, sizeof(datamessage), 0,
(sockaddr*)&serverAddr, sizeof(SOCKADDR_IN));
    arrive++;
    cout << Get_Time() << "    " << "[Send] seq = " << datamessage.SeqNum
<< ", checknum = " << datamessage.checkNum << endl;
    cout << Get_Time() << "    " << "[window] window size: " <<
window_size << ", has sent but not received confirmation: " << (arrive - start)
<< endl;

    ReleaseMutex(mutex);

}

if (resend || clock() - messagestart > MAX_WAIT_TIME) {
    if (resend)
        Print("Fast Retransmit...");

    WaitForSingleObject(mutex, INFINITE);

    for (int i = 0; i < arrive - start; i++) {
        Message resendMsg = messageBuffer.front();
        sendto(clientSocket, (char*)&resendMsg, sizeof(resendMsg), 0,
(sockaddr*)&serverAddr, sizeof(SOCKADDR_IN));
        cout << Get_Time() << "    " << "[Resend] seq = " <<
resendMsg.SeqNum << endl;

        messageBuffer.push(resendMsg);
        messageBuffer.pop();
    }
    ReleaseMutex(mutex);
}

```

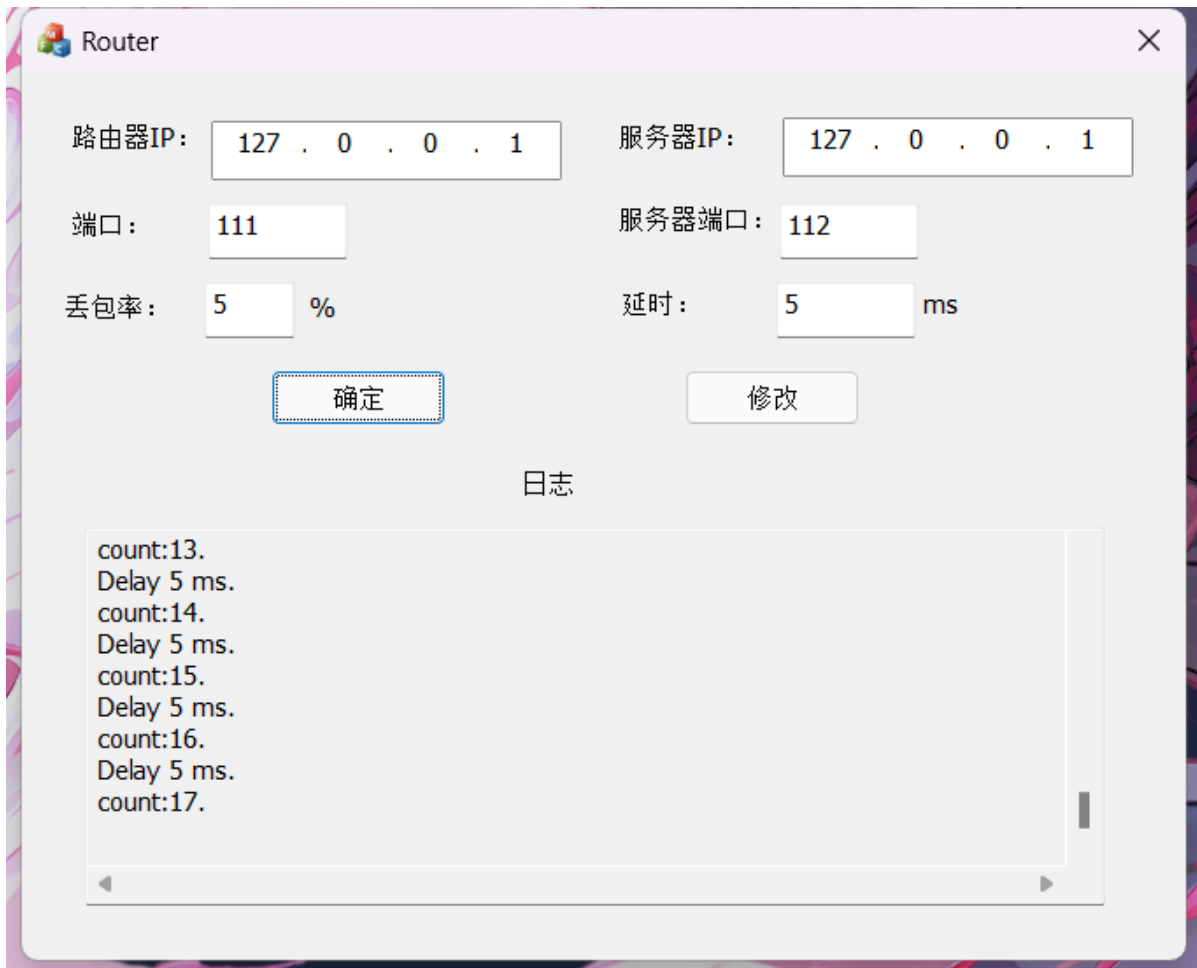
```
        messagestart = clock();  
        resend = false;  
    }  
  
    if (finish == true)  
        break;  
}  
CloseHandle(hThread);
```

## 实验结果

端口设置：

```
ServerPORT = 112  
ClientPORT = 110  
RouterPORT = 111
```

Router设置：



The image shows a Windows-style dialog box titled "Router" with a close button (X) in the top right corner. The dialog contains several input fields for configuration:

- 路由器IP:** 127 . 0 . 0 . 1
- 服务器IP:** 127 . 0 . 0 . 1
- 端口:** 111
- 服务器端口:** 112
- 丢包率:** 5 %
- 延时:** 5 ms

Below the input fields are two buttons: "确定" (OK) and "修改" (Modify).

At the bottom of the dialog is a section labeled "日志" (Log) containing a text area with the following text:

```
count:13.  
Delay 5 ms.  
count:14.  
Delay 5 ms.  
count:15.  
Delay 5 ms.  
count:16.  
Delay 5 ms.  
count:17.
```

测试文件传输

1.jpg

Microsoft Visual Studio 调试

2024-12-04 23:13:34 127 full load data message received successfully.  
[Send]seq = 130  
2024-12-04 23:13:34 [Send]seq = 129, ack = 130  
2024-12-04 23:13:34 128 full load data message received successfully.  
[Send]seq = 131  
2024-12-04 23:13:34 [Send]seq = 130, ack = 131  
2024-12-04 23:13:34 129 full load data message received successfully.  
[Send]seq = 132  
2024-12-04 23:13:39 [Send]seq = 131, ack = 132  
2024-12-04 23:13:39 130 full load data message received successfully.  
[Send]seq = 133  
2024-12-04 23:13:39 [Send]seq = 132, ack = 133  
2024-12-04 23:13:39 131 full load data message received successfully.  
[Send]seq = 134  
2024-12-04 23:13:39 [Send]seq = 133, ack = 134  
2024-12-04 23:13:39 132 full load data message received successfully.  
[Send]seq = 135  
2024-12-04 23:13:39 [Send]seq = 134, ack = 135  
2024-12-04 23:13:39 Not full load data message received successfully.  
2024-12-04 23:13:39 File has written successfully.  
2024-12-04 23:13:39 First Handwave succeeded.(Receive)  
2024-12-04 23:13:39 Second Handwave succeeded.(Send)  
2024-12-04 23:13:39 Third Handwave succeeded.(Send)  
2024-12-04 23:13:39 Forth Handwave succeeded.(Receive)  
2024-12-04 23:13:39 Server Connection has closed.

D:\Desktop\lab3-Server\Server\Server.exe (进程 11916)已退出, 代码为 0 (0x0).  
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。  
按住任意键关闭此窗口。 . . .

Microsoft Visual Studio 调试

2024-12-04 23:13:34 [Window] Window Residual Size: 26, has sent but not received confirmation: 6  
[Receive] ack = 130  
2024-12-04 23:13:34 [Window] Window Residual Size: 27, has sent but not received confirmation: 5  
[Receive] ack = 131  
2024-12-04 23:13:34 [Window] Window Residual Size: 28, has sent but not received confirmation: 4  
[Resend] seq = 132  
2024-12-04 23:13:39 [Resend] seq = 133  
2024-12-04 23:13:39 [Resend] seq = 134  
2024-12-04 23:13:39 [Receive] ack = 132  
2024-12-04 23:13:39 [Window] Window Residual Size: 29, has sent but not received confirmation: 3  
[Receive] ack = 133  
2024-12-04 23:13:39 [Window] Window Residual Size: 30, has sent but not received confirmation: 2  
[Receive] ack = 134  
2024-12-04 23:13:39 [Window] Window Residual Size: 31, has sent but not received confirmation: 1  
[Receive] ack = 135  
2024-12-04 23:13:39 [Window] Window Residual Size: 32, has sent but not received confirmation: 0  
Total transfer time: 48049ms  
Average throughput: 45.9627 bytes/ms  
First Handwave succeeded.(Send)  
2024-12-04 23:13:39 Second Handwave succeeded.(Receive)  
2024-12-04 23:13:39 Third Handwave succeeded.(Receive)  
2024-12-04 23:13:39 Forth Handwave succeeded.(Send)  
2024-12-04 23:13:39 Client in TIME\_WAIT state  
2024-12-04 23:13:49 Client successfully closed connection!

D:\Desktop\lab3-Client\Client\Debug\Client.exe (进程 20928)已退出, 代码为 0 (0x0).  
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。  
按住任意键关闭此窗口。 . . .

1.jpg 属性

常规 安全 属性修改 详细信息 以前的版本

1.jpg

文件类型: JPG 图片文件 (.jpg)  
打开方式: WPS 图片 更改(C)...  
位置: D:\Desktop\lab3-Server\Server  
大小: 1.77 MB (1,857,353 字节)  
占用空间: 1.77 MB (1,859,584 字节)  
创建时间: 2024年12月4日, 23:13:39  
修改时间: 2024年12月4日, 23:13:39  
访问时间: 2024年12月4日, 23:13:39  
属性: ☐ 只读(R) ☐ 隐藏(H) 高级(D)...  
确定 取消 应用(A)

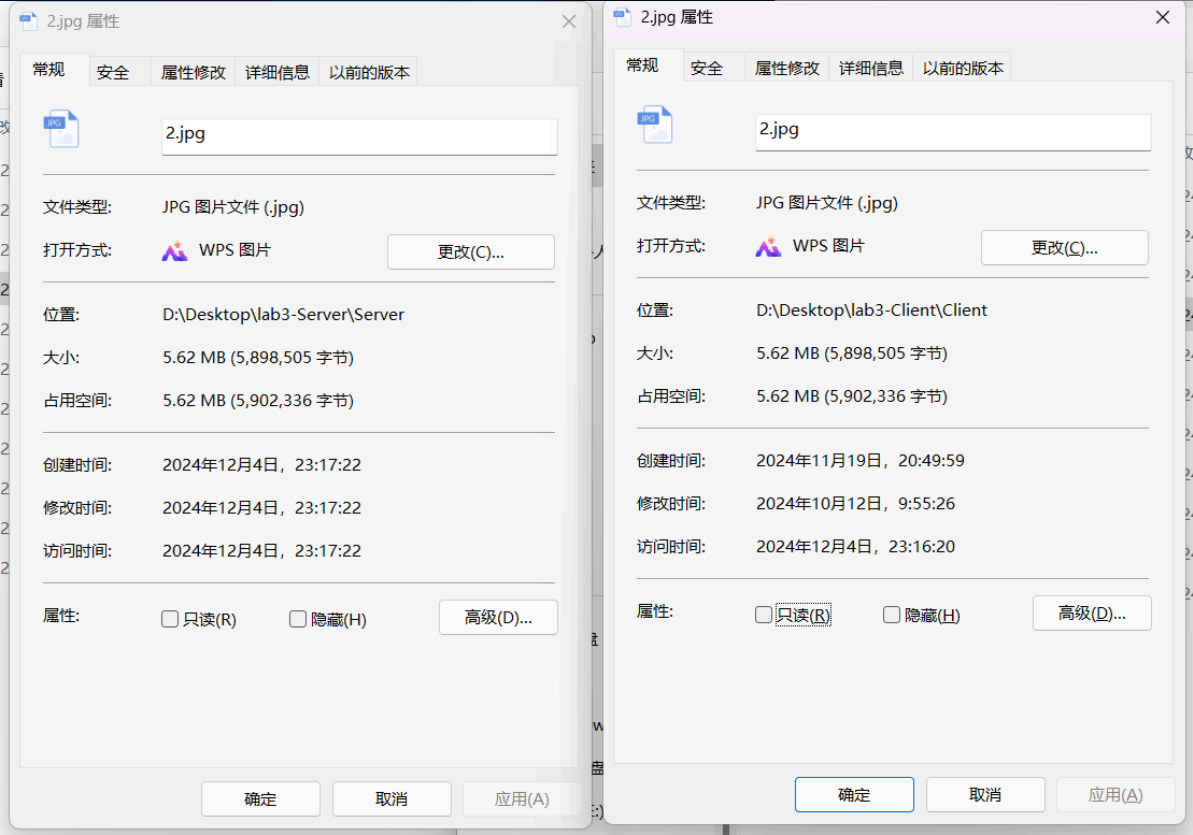
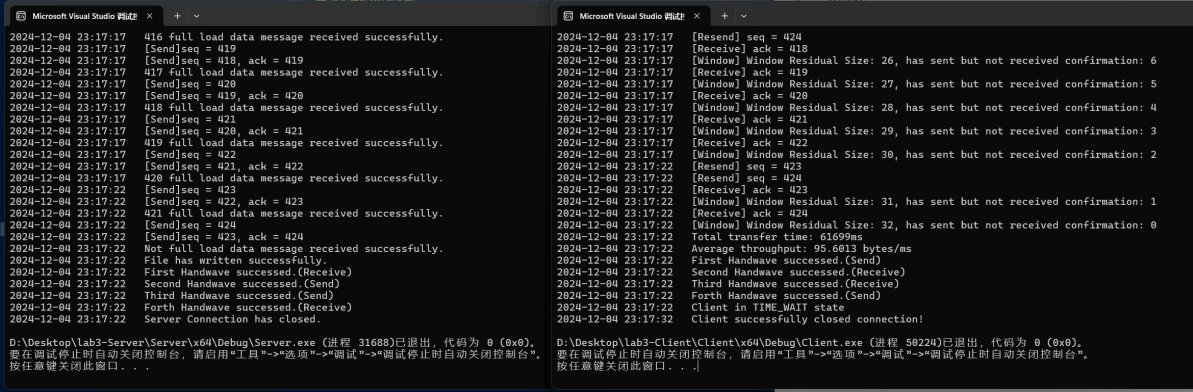
1.jpg 属性

常规 安全 属性修改 详细信息 以前的版本

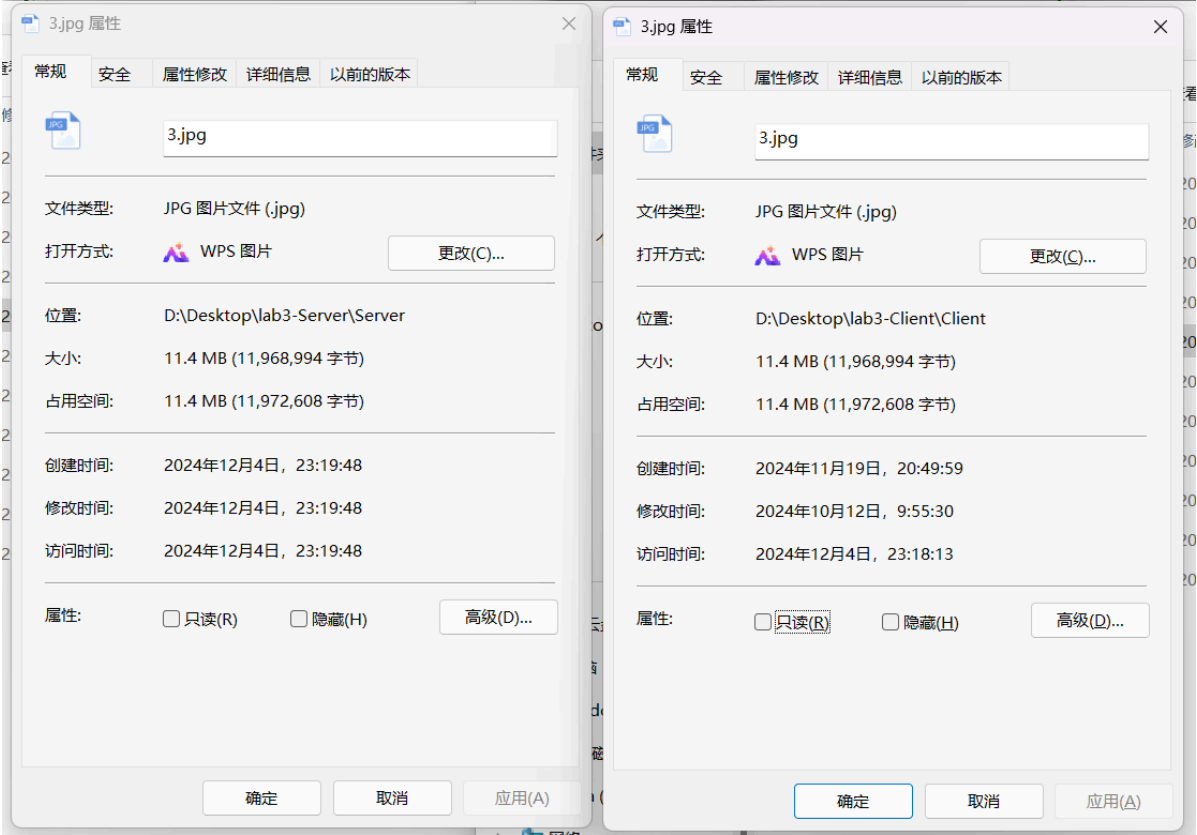
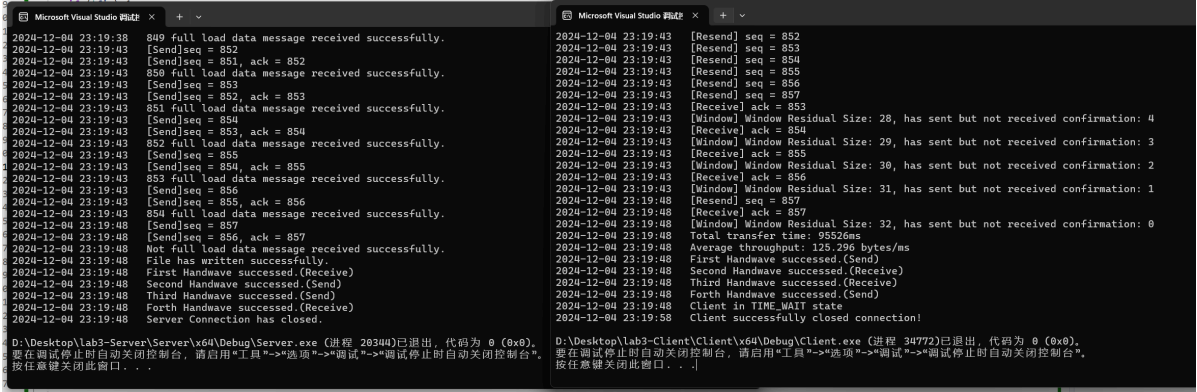
1.jpg

文件类型: JPG 图片文件 (.jpg)  
打开方式: WPS 图片 更改(C)...  
位置: D:\Desktop\lab3-Client\Client  
大小: 1.77 MB (1,857,353 字节)  
占用空间: 1.77 MB (1,859,584 字节)  
创建时间: 2024年11月19日, 20:49:59  
修改时间: 2024年10月12日, 9:55:28  
访问时间: 2024年12月4日, 23:12:59  
属性: ☐ 只读(R) ☐ 隐藏(H) 高级(D)...  
确定 取消 应用(A)

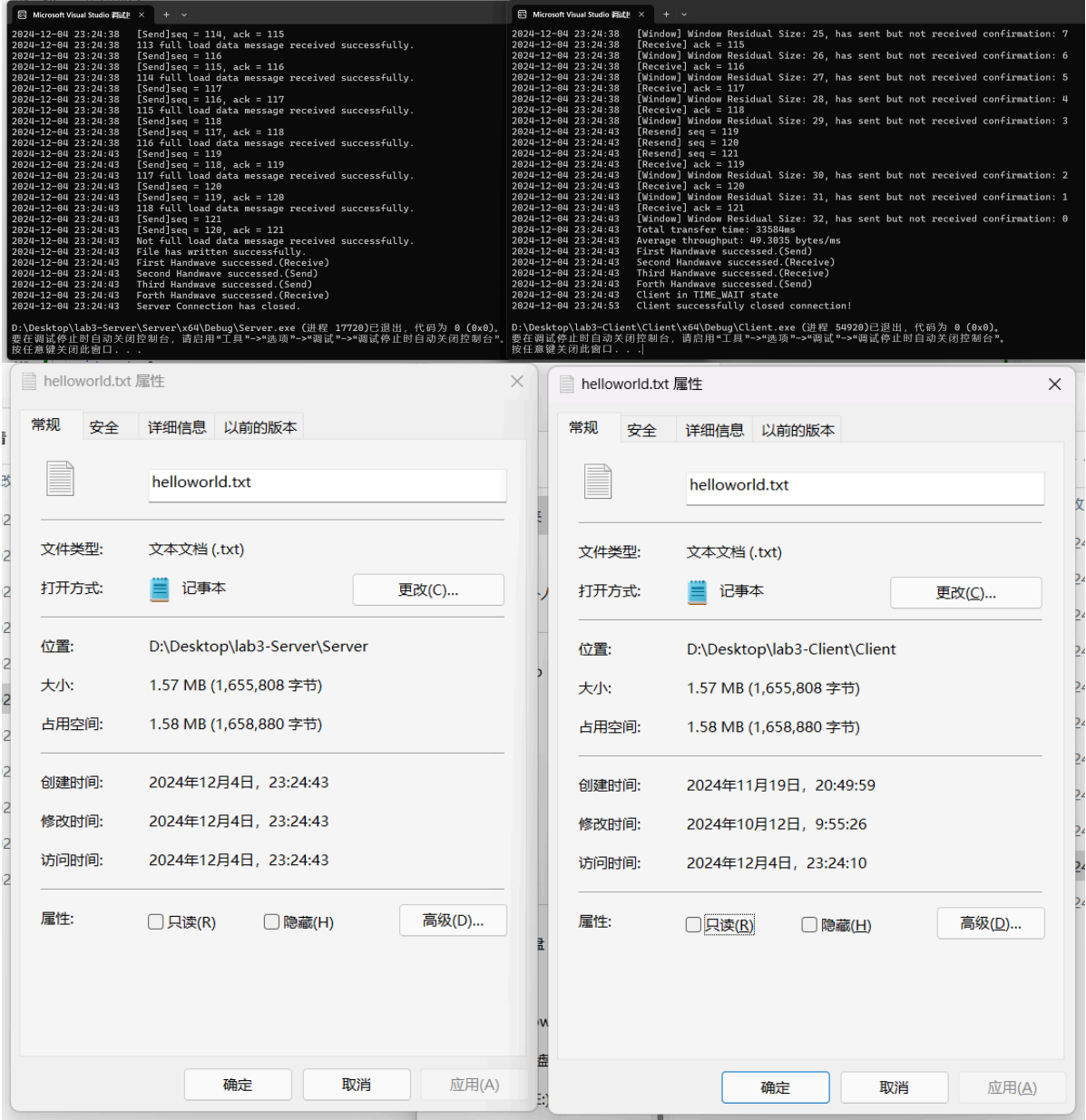
2.jpg



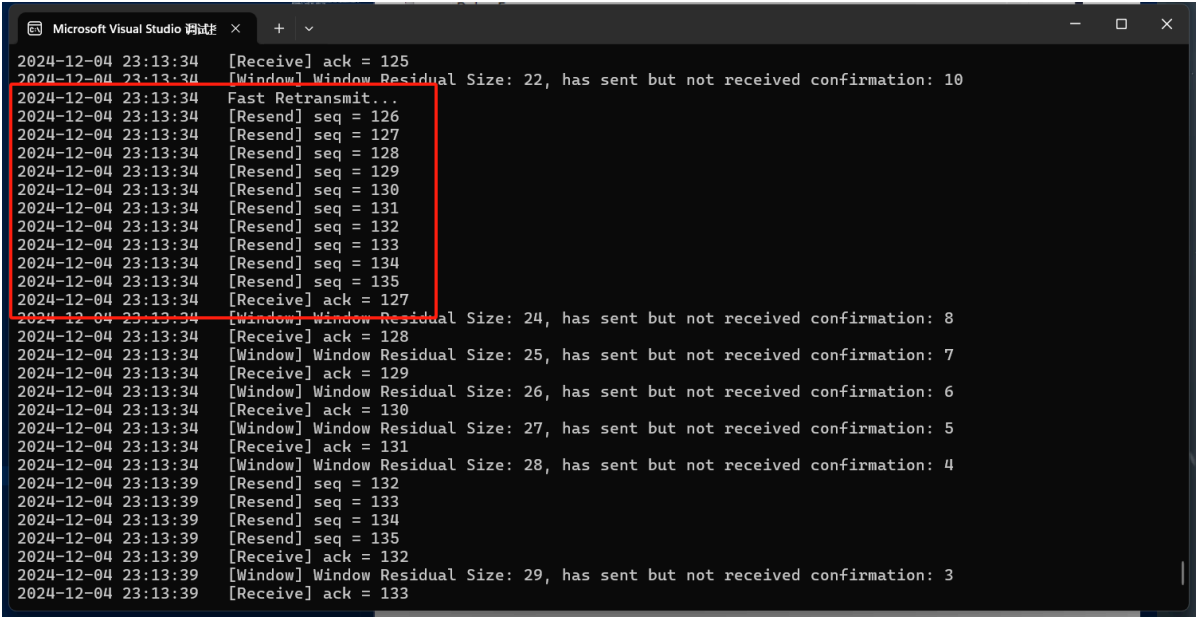
3.jpg



helloworld.txt



快速重传 超时重传



## 实验反思

---

在本次实验中，我实现了基于滑动窗口的可靠传输协议。通过实验，我更加了解了滑动窗口协议的原理和实现方式，掌握了如何利用滑动窗口协议实现可靠传输。

同时，我还学习了如何使用多线程来实现并行处理，提高程序的效率和响应速度。在实验中，由于 Client 端需要同步发送和接收，所以根据实验一中的经历，我选择使用双线程来解决这个问题。同时加锁来保证线程安全。